

EXERCISE I

With reference to the Strassen method described in the lecture

I.a) Explain why this method can be useful in neural networks computations

The Strassen's method is useful for reducing the complexity of matrix multiplication from $O(N^3)$ to $O(N^{2.8})$, which is the core computational kernel of most DNNs

Cons: requires significantly more memory

I.b) Compute the number of multiplications and additions needed for computing the product of two 4x4 matrices, using the baseline method and using Strassen (Use the recursive formula)

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix},$$

with $A_{ij}, B_{ij}, C_{ij} \in \text{Mat}_{2^{n-1} \times 2^{n-1}}(\mathcal{R})$. The naive algorithm would be:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}.$$

An ordinary matrix multiplication with $N \times N$ matrices requires N^3 multiplications and $N^3 - N^2$ additions

Given $N=4$:

Naïve mat-mul ops:

- $\text{ADD} = N^3 - N^2 = 64 - 16 = 48$
- $\text{MUL} = N^3 = 64$

$\text{TOT_naive} = 2 N^3 - N^2 = 112 \text{ ops}$

The Strassen algorithm defines instead new matrices:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22});$$

$$M_2 = (A_{21} + A_{22})B_{11};$$

$$M_3 = A_{11}(B_{12} - B_{22});$$

$$M_4 = A_{22}(B_{21} - B_{11});$$

$$M_5 = (A_{11} + A_{12})B_{22};$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12});$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22}),$$

using only 7 multiplications (one for each M_k) instead of 8. We may now express the C_{ij} in terms of M_k :

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{bmatrix}.$$

MUL(N), ADD(N) – multiplication/addition of matrices of size NxN

$$\text{MUL}(N) = 7 * \text{MUL}(N / 2) + 18 * \text{ADD}(N/2)$$

$$\text{MUL}(4) = 7 * \text{MUL}(2) + 18 * \text{ADD}(2)$$

$$\text{MUL}(2) = 7 * \text{MUL}(1) + 18 * \text{ADD}(1) = 7\text{MUL} + 18\text{ADD}$$

$$\text{Tot_strassen: } 7 * (7\text{MUL} + 18\text{ADD}) + 18 * 4\text{ADD} = 49\text{MUL} + 198\text{ADD}$$

I.c) Compute the speedup of Strassen with respect to the baseline matrix multiplication for matrix size 32x32 (Use the asymptotic formulas)

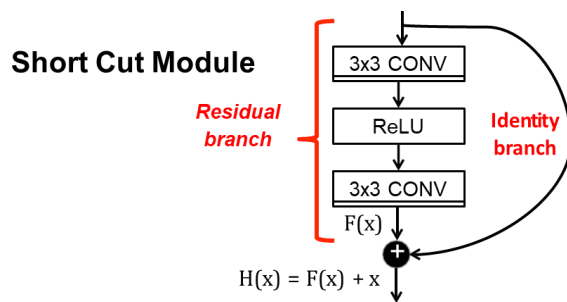
With a matrix size of N:

$$\text{Complexity naïve: } N^3 = 32^3 = 32768$$

$$\text{Complexity Strassen: } N^{2.81} = 16961$$

$$\text{Speedup} = 32768 / 16961 = 1.93x$$

EXERCISE II



Consider the shortcut module shown above with the following characteristics: 64 input and output channels for each convolution (spatial size 30×30). Padding with zeros, stride 1.

II.a) Compute the total number of operations for the entire shortcut module, assuming that ReLU requires one compare operation. Also consider the add operation needed for the merge of the identity branch.

$$\text{Conv1_MAC} = K * K * \text{Cin} * \text{Cout} * \text{Wout} * \text{Hout} = 3 * 3 * 64 * 64 * 30 * 30 = 33177600 \text{ MAC}$$

$$\text{Relu_ops} = \text{Cout} * \text{Wout} * \text{Hout} = 64 * 30 * 30 = 57600 \text{ CMP}$$

$$\text{Conv2_MAC} = K * K * \text{Cin} * \text{Cout} = 3 * 3 * 64 * 64 * 30 * 30 = 33177600 \text{ MAC}$$

$$\text{Sum_ops} = \text{Cout} * \text{Wout} * \text{Hout} = 64 * 30 * 30 = 57600 \text{ MAC}$$

$$\text{Tot_ops} = 33177600 * 2 \text{ MAC} + 57600 * 2 \text{ CMP} = 66'470'400 \text{ OPS}$$

II.b) Compute the total storage needed for weights (8bit per element)

$$\text{Conv1_size} = 3 * 3 * 64 * 64 = 36864 \text{ B}$$

$$\text{Conv2_size} = 3 * 3 * 64 * 64 = 36864 \text{ B}$$

$$\text{Tot size} = 73'728 \text{ B}$$

II.c) Compute the total storage needed for activations (16bits per element).

Each element is 2 Bytes

Solution 1: if we don't have any in-place operation

$$\text{Input} = (\text{ch} * \text{w} * \text{h}) * 2\text{B} = (64 * 30 * 30) * 2\text{B} = 115200 \text{ B}$$

$$\text{Out conv1} = (\text{ch} * \text{w} * \text{h}) * 2\text{B} = (64 * 30 * 30) * 2\text{B} = 115200 \text{ B}$$

$$\text{Out relu} = (\text{ch} * \text{w} * \text{h}) * 2\text{B} = (64 * 30 * 30) * 2\text{B} = 115200 \text{ B}$$

$$\text{Out conv2} = (\text{ch} * \text{w} * \text{h}) * 2\text{B} = (64 * 30 * 30) * 2\text{B} = 115200 \text{ B}$$

$$\text{Out add} = (\text{ch} * \text{w} * \text{h}) * 2\text{B} = (64 * 30 * 30) * 2\text{B} = 115200 \text{ B}$$

$$\text{tot} = 2304000 \text{ B}$$

Solution 2: if we have in place operations we only need to store maximum 3 activations at the time (Relu overwrites output conv1 and sum overwrites output conv2)

$$\text{Tot} = 115200 * 3 = 345600 \text{ B}$$

EXERCISE III

A parallel cluster of RISC-V processors is computing a sequence of two **3x3** convolutional layers, each with RELU and max-pooling (2x2 window) **with stride 2**. The input **spatial** size is 64x64. The first layer takes as an input 32 channels and outputs 64 channels. The second layer produces 100 output channels. Inputs and outputs are quantized to 16 bits and weights are quantized to 8 bits. Padding with zeros is used **on the convolutions**. Assume that on average a RISC-V processor can perform a MAC every 4 cycles, a ReLU every 3 cycles and a compare every 3 cycles. Assume that the computation is parallelized assigning output channels to be computed to processors, trying to maximally balance the workload. After each sub-layer (CONV, RELU, MAXPOOL, CONV, RELU, MAXPOOL), the processors synchronize (join) with a barrier

III.a) Compute the minimal memory needed to store the entire double-layer in L1. You can assume that the **RELUs are computed in place.**

Sequence: Input – Conv1 – Relu1 – maxpool1 – conv2 -relu2 - maxpool 2

Parameters=

- $\text{in_conv1} = w * h * ch = 64 * 64 * 32 = 131072$
- $\text{weights_conv1} = ch_in * ch_out * K * K = 32 * 64 * 3 * 3 = 18432$
- $\text{out_conv1} = w * h * ch = 64 * 64 * 64 = 262144$
- $\text{out_pool1} = w * h * ch = 32 * 32 * 64 = 65536$
- $\text{weights_conv2} = ch_in * ch_out * K * K = 64 * 100 * 3 * 3 = 57600$
- $\text{out_conv2} = w * h * ch = 32 * 32 * 100 = 102400$
- $\text{out_pool2} = w * h * ch = 16 * 16 * 100 = 25600$

Total_L1 = (weights_conv1 + weights_conv2)*1B + (in_conv1 + out_conv1 + out_pool1 + out_conv2 + out_pool2)*2B =
= (18432 + 57600)*1B + (131072 + 262144 + 65536 + 102400 + 25600)*2B = **1'249'536 B**

III.b) Compute the number of cycles to process the two layers on a single processor.

A maxpool needs

- Compare_per_pooling window = $(2*2-1) = 3$
- Compare_per_1_output_channel = $(2*2-1)*W_{out}*H_{out}$
- Compare_for_all_output_channel = $(2*2-1)*W_{out}*H_{out}*C_{out}$

Ops(1core)=

- $\text{conv1} = K * K * C_{in} * C_{out} * W_{out} * H_{out}$ MAC = $3 * 3 * 32 * 64 * 64 * 64$ MAC = 75497472 MAC
- $\text{Relu1} = C_{out} * W_{out} * H_{out}$ MAC = $64 * 64 * 64$ RELU = 262144 ReLU
- $\text{pool1} = 3 * W_{out} * H_{out} * C_{out}$ CMP = $3 * 32 * 32 * 64$ CMP = 196608 COMP

- $\text{conv2} = K * K * \text{Cin} * \text{Cout} * \text{Wout} * \text{Hout} \text{ MAC} = 3 * 3 * 64 * 100 * 32 * 32 \text{ MAC} = 58982400 \text{ MAC}$
- $\text{Relu2} = \text{Cout} * \text{Wout} * \text{Hout} \text{ MAC} = 100 * 32 * 32 \text{ RELU} = 102400 \text{ ReLU}$
- $\text{Pool2} = 3 * \text{Wout} * \text{Hout} * \text{Cout} \text{ CMP} = 3 * 16 * 16 * 100 \text{ CMP} = 76800 \text{ COMP}$

$$\text{Cycles} = (\text{conv1} + \text{conv2}) * 4 + (\text{Relu1} + \text{Relu2}) * 3 + (\text{pool1} + \text{pool2}) * 3 =$$

$$(75497472 + 58982400) * 4 + (262144 + 102400) * 3 + (196608 + 76800) * 3 = 539'833'344$$

III.c) Compute the speedup compared to sequential when the cluster has 8 cores.

Ops(8core)=

- $\text{conv1} = K * K * \text{Cin} * \text{round_ceil}(\text{Cout}/8) * \text{Wout} * \text{Hout} \text{ MAC} = 3 * 3 * 32 * (8) * 64 * 64 \text{ MAC} = 9437184 \text{ MAC}$
- $\text{Relu1} = \text{round_ceil}(\text{Cout}/8) * \text{Wout} * \text{Hout} \text{ MAC} = 8 * 64 * 64 \text{ RELU} = 32768 \text{ ReLU}$
- $\text{pool1} = 3 * \text{Wout} * \text{Hout} * \text{round_ceil}(\text{Cout}/8) \text{ CMP} = 3 * 32 * 32 * 8 \text{ CMP} = 24576 \text{ COMP}$
- $\text{conv2} = K * K * \text{Cin} * \text{round_ceil}(\text{Cout}/4) * \text{Wout} * \text{Hout} \text{ MAC} = 3 * 3 * 64 * \text{ceil}(12.5) * 32 * 32 \text{ MAC} = 3 * 3 * 64 * (13) * 32 * 32 \text{ MAC} = 7667712 \text{ MAC}$
- $\text{Relu2} = \text{round_ceil}(\text{Cout}/8) * \text{Wout} * \text{Hout} \text{ MAC} = 13 * 32 * 32 \text{ RELU} = 13312 \text{ ReLU}$
- $\text{Pool2} = 3 * \text{Wout} * \text{Hout} * \text{round_ceil}(\text{Cout}/8) \text{ CMP} = 3 * 16 * 16 * 13 \text{ CMP} = 9984 \text{ COMP}$

$$\text{Ops} = (\text{conv1} + \text{conv2}) * 4 + (\text{Relu1} + \text{Relu2}) * 3 + (\text{pool1} + \text{pool2}) * 3 =$$

$$(9437184 + 7667712) * 4 + (32768 + 13312) * 3 + (24576 + 9984) * 3 = 68661504$$

$$\text{Speedup} = 539833344 / 68661504 = 7.86x$$